

ΚΑΤΑΝΕΜΗΜΕΝΑ ΣΥΣΤΗΜΑΤΑ



Ακαδημαϊκό έτος 2025-2026

Ιωάννινα

ΠΕΡΙΕΧΟΜΕΝΑ

Περιεχόμενα

ΚΕΦΑΛΑΙΟ 1: Περιβάλλον υλοποίησης άσκησης και ανάλυση ζητουμένων.....	2
ΚΕΦΑΛΑΙΟ 2: Υλοποίηση άσκησης.....	3
ΚΕΦΑΛΑΙΟ 3: Τελικά αποτελέσματα στο τερματικό.....	19

ΚΕΦΑΛΑΙΟ 1: Περιβάλλον υλοποίησης άσκησης και ανάλυση ζητουμένων

Για την υλοποίηση της άσκησης χρησιμοποιήσαμε μία εικονική μηχανή στην οποία είναι εγκατεστημένα τα Linux Debian 12 με Linux kernel 6.1.0-20-amd64. Στην εικονική μηχανή παρέχονται 4 νήματα και 8 GB RAM και 28 GB αποθηκευτικού χώρου. Ο υπολογιστής ξενιστής έχει έναν τετραπύρνηο Intel Core-i7 7700HQ, 32GB RAM LPDDR4 @ 2400 MT/s και η εικονική μηχανή έτρεχε από έναν SATA 3 SSD Samsung 870 QVO με χωρητικότητα 1 TB. Για να τρέξει η εικονική μηχανή χρησιμοποιήθηκε ο hypervisor τύπου 2 VMWare Workstation 17 Player.

Για την ανάπτυξη του κώδικα χρησιμοποιήθηκε το Visual Studio Code.

Πίνακας 1Υλικό και λογισμικό για την υλοποίηση της άσκησης.

Επεξεργαστής	Intel Core-i7-7700HQ with 4C/8T
RAM	32 GB @ 2400 MT/s
SSD	SAMSUNG SSD 870 QVO 1TB
Linux version	Debian 12 (bookworm)
Linux Kernel	6.1.0-20amd64
Hypervisor	VMWare Workstation 17

Σκοπός της εργασίας ήταν η υλοποίηση ενός συστήματος συνομιλίας Client - Server σε γλώσσα C με χρήση sockets και νημάτων (threads). Το πρώτο μέρος της εργασίας απαιτούσε ο εξυπηρετητής να μπορεί να απαντήσει σε μήνυμα που έλαβε από τον πελάτη με χρήση ενός νήματος ανά διεργασία. Στο δεύτερο μέρος ο πελάτης μπορεί να στέλνει μηνύματα στον εξυπηρετητή χωρίς να περιμένει πρώτα απάντηση όπως συνέβαινε στο πρώτο μέρος, αυτό συμβαίνει με την αξιοποίηση δύο νημάτων από τον πελάτη. Το τρίτο μέρος της εργασίας απαιτούσε ο εξυπηρετητής να διαχειρίζεται πολλούς πελάτες ταυτόχρονα, να λαμβάνει μηνύματα από αυτούς και να τα προωθεί σε όλους τους υπόλοιπους συνδεδεμένους πελάτες χρησιμοποιώντας πολυνηματικό εξυπηρετητή. Επίσης, κάθε πελάτης μπορεί να ορίζει ένα μοναδικό όνομα για τον εαυτό του που θα φαίνεται στους υπόλοιπους πελάτες όταν στέλνει μηνύματα.

ΚΕΦΑΛΑΙΟ 2: Υλοποίηση άσκησης

- Έκδοση#1: αίτηση-απάντηση με ένα νήμα ανά διεργασία

Το σύστημα αποτελείται από δύο προγράμματα, τον εξυπηρετητή (Server) και τον πελάτη (Client) και υλοποιήθηκε πρωτόκολλο αίτησης - απάντησης μεταξύ ενός πελάτη και του εξυπηρετητή. Ο πελάτης στέλνει ένα μήνυμα και στη συνέχεια περιμένει την απάντηση του εξυπηρετητή για να στείλει εκ νέου μήνυμα. Αντίστοιχα, ο εξυπηρετητής αφού λάβει ένα μήνυμα από τον πελάτη τότε μόνο μπορεί να απαντήσει σε αυτόν με ένα μήνυμα. Στον κώδικα του πελάτη προστέθηκε μια κλήση της `recvMessage()` αμέσως μετά την `sendMessage()` ώστε μετά την αποστολή του μηνύματος ο πελάτης μπλοκάρει μέχρι να πάρει απάντηση από τον εξυπηρετητή. Αν η `recvMessage()` επιστρέψει 0 σημαίνει ότι ο εξυπηρετητής αποσυνδέθηκε και ο πελάτης τερματίζει. Ακολουθούν διάφορα κομμάτια του κώδικα.

Κώδικας για Πελάτη:

```
59 |         if (!EOF_found) {
60 |             if (sendMessage(socket_fd, msg) == -1) {
61 |                 close(socket_fd);
62 |                 perror("sendMessage failure");
63 |                 exit(EXIT_FAILURE);
64 |             }
65 |             ssize_t read_bytes = recvMessage(socket_fd, msg, MAX_MSG_LEN);
66 |
67 |             if (read_bytes == 0){
68 |                 printf("Server disconnected!\n");
69 |                 break;
70 |             }
71 |
72 |             snprintf(print_msg, sizeof(print_msg), "Server> %s\n", msg);
73 |             printMsg(stdout, print_msg);
74 |
75 |             strcpy(msg, "");
76 |         }
77 |     } while (!EOF_found);
78 |
79 |     close(socket_fd);
80 |
81 |     return 0;
82 | }
```

Στον παραπάνω κώδικα ο πελάτης μέχρι να βρεθεί το τέλος του αρχείου (End Of File) στέλνει μήνυμα στον εξυπηρετητή, περιμένει απάντηση. Όταν λάβει απάντηση από τον εξυπηρετητή συνεχίζει περιμένοντας είσοδο από το πληκτρολόγιο, η οποία αν δεν είναι το τέλος του αρχείου (EOF) στέλνεται στον εξυπηρετητή Κώδικας για εξυπηρετητή:

```

17 int main() {
18     struct sockaddr_in addr;
19     int socket_fd;
20     int client_fd;
21     int socket_option;
22
23     char reply[MAX_MSG_LEN + 1]; //This is where server's response is stored
24
25     // data required to read the IP address of the connected client
26     struct sockaddr_in client_addr;
27     socklen_t client_addr_len = sizeof(client_addr);
28     char client_ip[INET_ADDRSTRLEN];

```

Ο εξυπηρετητής αποθηκεύει στον πίνακα “reply” το αλφαριθμητικό που στέλνει ως απάντηση στον πελάτη.

```

100     strcpy(msg, "");
101     printMsg(stdout, "Reply> ");
102
103     do {
104         if (fgets(reply, sizeof(reply), stdin) == NULL) {
105             break;
106         }
107         reply[strcspn(reply, "\n")] = '\0';
108     } while (strlen(reply) == 0);
109     if (sendMessage(client_fd, reply) == -1) {
110         perror("sendMessage failure");
111         close(client_fd);
112         close(socket_fd);
113         exit(EXIT_FAILURE);
114     }
115     strcpy(reply, "");
116 }
117
118 close(socket_fd);
119 return 0;
120 }

```

Ο εξυπηρετητής διαβάζει είσοδο από το πληκτρολόγιο, στο τέλος της εισόδου αντικαθιστά τον χαρακτήρα “\0” με τον χαρακτήρα αλλαγής γραμμής “\n” και στέλνει την τροποποιημένη είσοδο στον πελάτη. Σε περίπτωση αποτυχίας κλείνει τα sockets και τερματίζει, ενώ στο τέλος του βρόχου το αλφαριθμητικό reply καθαρίζεται.

- Έκδοση#2: αίτηση-απάντηση με δύο νήματα ανά διεργασία

Το σύστημα αποτελείται από δύο προγράμματα, τον εξυπηρετητή (Server) και τον πελάτη (Client). Στη προκειμένη περίπτωση ο πελάτης μπορεί να στέλνει μηνύματα στον εξυπηρετητή και να λαμβάνει μηνύματα από αυτόν, όπως και ο εξυπηρετητής μπορεί να στέλνει μηνύματα στον πελάτη και να λαμβάνει από αυτόν. Η ειδοποιός διαφορά με τη πρώτη έκδοση είναι πως και ο εξυπηρετητής και ο πελάτης έχουν από δύο νήματα ο καθένας με το ένα νήμα να είναι υπεύθυνο για τη λήψη μηνυμάτων και το άλλο για την αποστολή τους. Με τη χρήση

ξεχωριστού μηνύματος για τη λήψη και ξεχωριστού για την αποστολή μηνυμάτων επιτυγχάνουμε το να μπορεί οποιοδήποτε μέλος της σύνδεσης να στείλει και να λάβει μήνυμα οποτεδήποτε το επιθυμεί, δίχως μετά την αποστολή ενός μηνύματος να χρειάζεται να λάβει πρώτα μήνυμα από το άλλο άκρο, για να μπορεί να ξαναστείλει μήνυμα, και ούτε να είναι απαραίτητο το να στείλει πρώτα μήνυμα, για να μπορεί να λάβει. Ακολουθούν διάφορα κομμάτια του κώδικα.

Κώδικας για Πελάτη:

```
15 struct infoPassed {
16     int socket_fd;
17     int* EOF_found;
18 };
19
20 void* receive_thread(void* args){
21     int* EOF_found = ((struct infoPassed *)args)->EOF_found;
22     int socket_fd = ((struct infoPassed *)args)->socket_fd;
23     while (!(*EOF_found)){
24         char msg[MAX_MSG_LEN + 1];
25         char print_msg[MAX_PRINT_MSG_LEN + 1];
26         ssize_t read_bytes = recvMessage(socket_fd, msg, MAX_MSG_LEN);
27         if (read_bytes <= 0){
28             if (!(*EOF_found)){
29                 fprintf(stdout, "Server disconnected.\n");
30                 exit(EXIT_FAILURE);
31             }
32             break;
33         }
34         snprintf(print_msg, sizeof(print_msg), "Server> %s\n", msg);
35         printMsg(stdout, print_msg);
36         strcpy(msg, "");
37     }
38     return NULL;
```

Δημιουργήσαμε τη δομή τύπου struct infoPassed, για να παίρνει το νήμα που είναι υπεύθυνο για τη διαχείριση της πληροφορίας, η οποία στέλνεται από τον εξυπηρετητή, τον file descriptor για το socket και να επικοινωνεί με το κύριο νήμα για το πότε γράφεται το End Of File (EOF).

```
41 int main()
42 {
43     struct sockaddr_in addr;
44     int socket_fd;
45     char msg[MAX_MSG_LEN + 1];
46     char print_msg[MAX_PRINT_MSG_LEN + 1];
47     int EOF_found;
48
49     struct infoPassed infoPassed;
50
51     socket_fd = socket(AF_INET, SOCK_STREAM, 0);
52     if (socket_fd == -1) {
53         perror("socket failure");
54         exit(EXIT_FAILURE);
55     }
```

```

72     EOF_found = 0;
73     pthread_t receiver; //New thread for receiving messages
74     infoPassed.socket_fd = socket_fd;
75     infoPassed.EOF_found = &EOF_found;
76     if (pthread_create(&receiver, NULL, receive_thread, &infoPassed) != 0){
77         fprintf(stdout, "pthread_create failure");
78         exit(EXIT_FAILURE);
79     }
80
81     do {
82         do {
83             snprintf(print_msg, sizeof(print_msg), "> ");
84             printMsg(stdout, print_msg);
85
86             if(fgets(msg, sizeof(msg), stdin) == NULL) {
87                 EOF_found = 1;
88                 fprintf(stdout, "Terminating connection... Exiting\n");
89                 shutdown(socket_fd, SHUT_RDWR);
90                 pthread_join(receiver, NULL); //Wait for receiver thread to terminate
91                 close(socket_fd);
92                 break;
93             }
94
95             msg[strcspn(msg, "\n")] = '\0';
96         } while (!EOF_found && strlen(msg) == 0);
97
98         if (!EOF_found) {
99             if (sendMessage(socket_fd, msg) == -1) {
100                 close(socket_fd);
101                 fprintf(stdout, "sendMessage failure");
102                 exit(EXIT_FAILURE);
103             }
104         }
105     } while (!EOF_found);
106     return 0;

```

Δημιουργούμε τη δομή infoPassed και περνάμε σε αυτή το κατάλληλο socket και τη διεύθυνση της μνήμης για το flag του End Of File (EOF). Δημιουργούμε το νέο νήμα που είναι υπεύθυνο να λαμβάνει πληροφορία από τον εξυπηρετητή και του αναθέτουμε να εκτελεί την receive_thread(). Εν αντιθέσει με την προηγούμενη έκδοση της εργασίας, το κύριο νήμα είναι υπεύθυνο να διαβάζει απλά είσοδο από το πληκτρολόγιο και να την στέλνει στον εξυπηρετητή.

Κώδικας για εξυπηρετητή:

```

16     struct fileDescriptorsAndIP {
17         int socket_fd;
18         int client_fd;
19         char * client_ip;
20         int* EOF_found;
21     };

```

Χρησιμοποιούμε τη δομή fileDescriptorAndIP δίνουμε στο νήμα που είναι υπεύθυνο για τη συλλογή πληροφορίας που στέλνει ο πελάτης στον εξυπηρετητή. Το νήμα αυτό χρειάζεται να

γνωρίζει το ποιοι είναι οι file descriptors που χρησιμοποιούνται από τον εξυπηρετητή, η IP του client και το τέλος του αρχείου.

```
23 void* receive_thread(void* fds){
24     int socket_fd = ((struct fileDescriptorsAndIP *)fds)->socket_fd;
25     int client_fd = ((struct fileDescriptorsAndIP *) fds)->client_fd;
26     int* EOF_found = ((struct fileDescriptorsAndIP *)fds)->EOF_found;
27     char msg[MAX_MSG_LEN + 1];
28     char print_msg[MAX_PRINT_MSG_LEN + 1];
29     char * client_ip;
30     int client_ip_len = strlen(((struct fileDescriptorsAndIP *) fds)->client_ip);
31     client_ip = (char *) malloc(client_ip_len);
32     strncpy( client_ip , ((struct fileDescriptorsAndIP *) fds)->client_ip, client_ip_len );
33     while (1){
34         ssize_t read_bytes = recvMessage(client_fd, msg, MAX_MSG_LEN);
35         if (read_bytes == 0 && !(*EOF_found)) {
36             snprintf(print_msg, sizeof(print_msg), "-- User %s has left the conversation\n", client_ip);
37             printMsg(stdout, print_msg);
38             close(client_fd);
39             free(client_ip);
40             fprintf(stdout, "Client disconnected. Terminating\n");
41             exit(EXIT_FAILURE);
42         }
43         else if (read_bytes == -1 && !(*EOF_found)) {
44             perror("recvMessage failure");
45             close(client_fd);
46             close(socket_fd);
47             exit(EXIT_FAILURE);
48         } else if (read_bytes == -2 && !(*EOF_found)) {
49             snprintf(print_msg, sizeof(print_msg), "error: the message of the client cannot fit in the 'msg' buffer\n");
50             printMsg(stderr, print_msg);
51             close(client_fd);
52             close(socket_fd);
53             exit(EXIT_FAILURE);
54         }
55         if (*EOF_found) return NULL;
56         snprintf(print_msg, sizeof(print_msg), "\033[35mUser %s\033[0m> %s\n> ", client_ip, msg);
57         printMsg(stdout, print_msg);
58         strcpy(msg, "");
59     }
60 }
```

Σε περίπτωση που προκύψει σφάλμα το νήμα κλείνει τους file descriptors και τερματίζει το πρόγραμμα. Το client IP το χρειάζεται για να τυπώνει το ποιος πελάτης στέλνει το μήνυμα, καθώς η ταυτότητά του είναι η IP του, και έχοντας πρόσβαση στη μεταβλητή EOF_found δε εκτελεί ξανά τον βρόγχο όταν το κύριο νήμα διαβάζει EOF από το πληκτρολόγιο.

```

62 int main() {
63     struct sockaddr_in addr;
64     struct fileDescriptorsAndIP fds;
65     int socket_option;
66
67     char reply[MAX_MSG_LEN + 1]; //
68     int EOF_found;

```

```

132 do {
133     if (fgets(reply, sizeof(reply), stdin) == NULL) {
134         EOF_found = 1;
135         shutdown(fds.client_fd, SHUT_RDWR);
136         close(fds.client_fd);
137         shutdown(fds.socket_fd, SHUT_RDWR);
138         close(fds.socket_fd);
139         fprintf(stdout, "\n");
140         break;
141     }
142     reply[strcspn(reply, "\n")] = '\0';
143 } while (strlen(reply) == 0 && EOF_found == 0);
144 if (EOF_found) break;
145 if (sendMessage(fds.client_fd, reply) == -1) {
146     perror("sendMessage failure");
147     close(fds.client_fd);
148     close(fds.socket_fd);
149     exit(EXIT_FAILURE);
150 }
151 strcpy(reply, "");
152
153 pthread_join(receiver, NULL);

```

Το κύριο νήμα είναι υπεύθυνο μόνο, για να λαμβάνει την είσοδο από το πληκτρολόγιο και να τη στέλνει στον πελάτη. Όπως και προηγουμένως, αν η είσοδος από το πληκτρολόγιο είναι EOF βγαίνει από τον βρόγχο και τερματίζει, μόνο που τώρα ενημερώνει και το νήμα που είναι υπεύθυνο για τη λήψη πληροφορίας από τον πελάτη ότι χρειάζεται να τερματίσει το πρόγραμμα μέσω της `EOF_found` και της `shutdown(fds.client_fd, SHUT_RDWR)`. Με τη `shutdown(fds.client_fd, SHUT_RDWR)` ενημερώνεται και ο πελάτης. Μετά τον βρόγχο αναμένει το νήμα το οποίο είναι υπεύθυνο για τη λήψη να τερματίσει, και ύστερα συνεχίζει. Σε περίπτωση σφάλματος αποστολής το νήμα τερματίζει το πρόγραμμα έχοντας κλείσει τους file descriptors.

- Έκδοση #3: αίτηση χωρίς απάντηση με πολυνηματικό εξυπηρετητή

Στην έκδοση 3 της εργασίας το σύστημα αποτελείται ξανά από δύο προγράμματα, τον εξυπηρετητή (Server) και τον πελάτη (Client), ο εξυπηρετητής υποστηρίζει την ταυτόχρονη σύνδεση πολλών πελατών, λαμβάνει μηνύματα από αυτούς και τα προωθεί στους υπόλοιπους συνδεδεμένους πελάτες. Ο εξυπηρετητής δημιουργεί socket TCP, κάνει bind στη θύρα της εργασίας, με χρήση `listen()` ακούει για νέες συνδέσεις, αποδέχεται νέους πελάτες καλώντας `accept()` και δημιουργεί ξεχωριστό thread για κάθε πελάτη. Ο πελάτης από την άλλη συνδέεται στον εξυπηρετητή, δημιουργεί ένα thread που λαμβάνει μηνύματα και επιτρέπει στο χρήστη να στέλνει μηνύματα από το κύριο νήμα.

Για την αποθήκευση των συνδεδεμένων πελατών χρησιμοποιήθηκε η δομή `clientsList` η οποία διατηρεί τα `file descriptors` όλων των συνδεδεμένων πελατών, το πλήθος των πελατών και κάνει `mutex` για ασφαλή πρόσβαση από πολλά `threads`. Επίσης, χρησιμοποιήθηκε η δομή `clientInfoPassed` για να μπορεί κάθε `thread` να έχει πρόσβαση σε όλες τις απαραίτητες πληροφορίες του αντίστοιχου πελάτη. Η δομή αυτή εισήχθη, καθώς ο εξυπηρετητής έπρεπε να διαχειρίζεται πολλούς πελάτες ταυτόχρονα με χρήση νημάτων και κάθε νήμα χρειάζεται πρόσβαση όχι μόνο σε δικά του δεδομένα, αλλά και σε κοινές πληροφορίες του εξυπηρετητή. Ακολουθεί ο κώδικας της δομής που βρίσκεται στο αρχείο `infoForReceivers.h`.

```
13 struct clientInfoPassed {
14     struct clientsList * allClients;
15     int *client_fd;
16     int *socket_fd;
17     pthread_t* threadAddress;
18     struct clientInfoPassed*** structureWithAllInfoPassed;
19 };
```

Παρακάτω φαίνεται ο πλήρης κώδικας της `infoForReceivers.h`:

```
9  #include "clientsList.h"
10
11 #ifndef GLOBAL_VAR
12 #define GLOBAL_VAR
13
14 extern pthread_mutex_t lockForAllClientsInfo;
15
16 #endif
17
18 struct clientInfoPassed {
19     struct clientsList * allClients;
20     int *client_fd;
21     int *socket_fd;
22     pthread_t* threadAddress;
23     struct clientInfoPassed*** structureWithAllInfoPassed;
24 };
25
26 int initializeAllClientsInfo(struct clientInfoPassed ** allClientsInfo[]);
27 int addReceiverThreadInfo(struct clientInfoPassed** allClientsInfo[], struct clientInfoPassed** clientInfoForAddition);
28 int removeReceiverThreadInfo(struct clientInfoPassed ** allClientsInfo[], struct clientInfoPassed** clientInfoForRemoval);
29 int destroyAllClientsInfo(struct clientInfoPassed ** allClientsInfo[]);
30 int numberOfRemainingClients(struct clientInfoPassed ** allClientsInfo[]);
```

Για να διαχειριστούμε τους πελάτες που συνδέονται και αποσυνδέονται δημιουργήσαμε το αρχείο `infoForReceivers.c` το οποίο περιέχει συναρτήσεις για τη διαχείριση της δομής τύπου `clientInfoPassed`. Η συνάρτηση `initializeAllClientsInfo()` αρχικοποιεί τη δομή. Η συνάρτηση `addReceiverThreadInfo()` προσθέτει ένα πελάτη στον πρώτο κενό χώρο του πίνακα, κλειδώνει το `mutex` ώστε δύο `threads` να μην αλλάζουν τον πίνακα ταυτόχρονα και αν βρει κενή θέση `NULL` αποθηκεύει εκεί το `clientInfoPassed`. Η `removeReceiverThreadInfo()` αφαιρεί τον πελάτη από τον πίνακα, κλείνει το `file descriptor` για αυτόν, κάνει αποδέσμευση της μνήμης που είχε δεσμευτεί για τον πελάτη με `free()`, και βάζει `NULL` στη θέση του πίνακα. Η συνάρτηση `numberOfRemainingClients()` καταμετρά το πόσοι `clients` υπάρχουν ακόμη στη δομή και επιστρέφει το πλήθος τους, ενώ η συνάρτηση `destroyAllClientsInfo()` καταστρέφει το `mutex` και αποδεσμεύει τον χώρο που χρησιμοποιεί η δομή στην οποία αποθηκεύτηκαν όλες οι

πληροφορίες που χρειάζονται τα νήματα που είναι υπεύθυνα για τη λήψη πληροφορίας από τους πελάτες.

Ακολουθεί ο κώδικας του αρχείου infoForReceivers.c:

```
9  #include "infoForReceivers.h"
10
11  pthread_mutex_t lockForAllClientsInfo;
12
13  int initializeAllClientsInfo(struct clientInfoPassed ** allClientsInfo[]){
14      pthread_mutex_init(&lockForAllClientsInfo, NULL);
15      for (int i = 0; i < MAX_CONNECTED_CLIENTS; i++){
16          (*allClientsInfo)[i] = NULL;
17      }
18      return 0;
19  }
20
21  int addReceiverThreadInfo(struct clientInfoPassed** allClientsInfo[], struct clientInfoPassed** clientInfoForAddition){
22      pthread_mutex_lock(&lockForAllClientsInfo);
23      for (int i = 0; i < MAX_CONNECTED_CLIENTS; i++){
24          if ((*allClientsInfo)[i] == NULL){
25              (*allClientsInfo)[i] = *clientInfoForAddition;
26              pthread_mutex_unlock(&lockForAllClientsInfo);
27              return 0;
28          }
29      }
30      pthread_mutex_unlock(&lockForAllClientsInfo);
31      fprintf(stderr, "Failed to add new client info in allClientsInfo");
32      return -1;
33  }
34
35  int removeReceiverThreadInfo(struct clientInfoPassed ** allClientsInfo[], struct clientInfoPassed** clientInfoForRemoval){
36      pthread_mutex_lock(&lockForAllClientsInfo);
37      for (int i = 0; i < MAX_CONNECTED_CLIENTS; i++){
38          if ((*allClientsInfo)[i] == (*clientInfoForRemoval)){
39              close((*clientInfoForRemoval)->client_fd);
40              removeClient((*clientInfoForRemoval)->allClients, *((*clientInfoForRemoval)->client_fd));
41              free((*clientInfoForRemoval)->client_fd);
42              free((*clientInfoForRemoval)->threadAddress);
43              free(*clientInfoForRemoval);
44              (*allClientsInfo)[i] = NULL;
45              pthread_mutex_unlock(&lockForAllClientsInfo);
46              return 0;
47          }
48      }
49      pthread_mutex_unlock(&lockForAllClientsInfo);
50      fprintf(stderr, "Failed to remove info for a client from allClientsInfo\n");
51      return -1;
52  }
53
54  int numberOfRemainingClients(struct clientInfoPassed ** allClientsInfo[]){
55      int j = 0;
56      pthread_mutex_lock(&lockForAllClientsInfo);
57      for (int i = 0; i < MAX_CONNECTED_CLIENTS; i++){
58          if ((*allClientsInfo)[i] != NULL) j++;
59      }
60      pthread_mutex_unlock(&lockForAllClientsInfo);
61      return j;
62  }
63
64  int destroyAllClientsInfo(struct clientInfoPassed ** allClientsInfo[]){
65      pthread_mutex_destroy(&lockForAllClientsInfo);
66      free(*allClientsInfo);
67      return 0;
68  }
```

Επιπρόσθετα, υλοποιήθηκαν οι συναρτήσεις sendNewMessage() και recvNewMessage() ώστε μαζί με το μήνυμα να αποστέλλεται και το όνομα του αποστολέα. Αναλυτικότερα, η

sendNewMessage() στέλνει αρχικά το μήκος του ονόματος , μετά το όνομα και στη συνέχεια το μήκος και το περιεχόμενο του μηνύματος. Η recvNewMessage() λαμβάνει τα δεδομένα με την ίδια ακριβώς σειρά, πρώτα διαβάζει το όνομα του αποστολέα και μετά το μήνυμα καθώς επίσης και περιλαμβάνει ελέγχους εγκυρότητας για το μέγεθος του ονόματος και του μηνύματος αντίστοιχα ώστε να αποφεύγονται buffer overflows και λανθασμένες αναγνώσεις. Παρακάτω φαίνονται οι υλοποιημένες συναρτήσεις sendNewMessage() και recvNewMessage(), οι οποίες βρίσκονται στο αρχείο msg.c:

```
56 ssize_t sendNewMessage(int fd, const char *msg, const char *sender)
57 {
58     size_t sender_len = strlen(sender);
59     uint32_t net_sender_len = htonl(sender_len);
60
61     size_t msg_len = strlen(msg);
62     uint32_t net_msg_len = htonl(msg_len);
63
64     size_t forSender = 0;
65     size_t forMessage = 0;
66
67     if (writeall(fd, &net_sender_len, sizeof(net_sender_len)) == -1){
68         perror("Sending sender's name has failed");
69         return -1;
70     }
71     forSender = writeall(fd, sender, sender_len);
72     writeall(0, NULL, 0);
73     if (writeall(fd, &net_msg_len, sizeof(net_msg_len)) == -1) {
74         perror("Sending message has failed");
75         return -1;
76     }
77     forMessage = writeall(fd, msg, msg_len);
78     writeall(0, NULL, 0);
79     return (forSender+forMessage);
80 }
```

```

81 ssize_t recvNewMessage(int fd, char *msg, size_t msg_max_len, char *sender, size_t sender_max_len)
82 {
83     uint32_t net_sender_len;
84     size_t sender_len;
85     ssize_t name_read;
86
87     name_read = readall(fd, &net_sender_len, sizeof(net_sender_len));
88     if (name_read <= 0) {
89         return name_read;
90     }
91     sender_len = ntohl(net_sender_len);
92
93     if (sender_len >= sender_max_len) {
94         return -30;
95     }
96
97     if (sender_len > MAX_CLIENT_NAME_LEN){
98         return -7;
99     }
100
101     name_read = readall(fd, sender, sender_len);
102     if (name_read <= 0) return name_read;
103
104     uint32_t net_msg_len;
105     size_t msg_len;
106     ssize_t nread;
107     nread = readall(fd, &net_msg_len, sizeof(net_msg_len));
108     if (nread <= 0) {
109         return nread;
110     }
111
112     msg_len = ntohl(net_msg_len);
113
114     if (msg_len >= msg_max_len) {
115         return -4;
116     }
117

```

```

118         if (msg_len > MAX_MSG_LEN) {
119             return -3;
120         }
121
122         nread = readall(fd, msg, msg_len);
123         if (nread <= 0) {
124             return nread;
125         }
126
127         msg[nread+name_read] = '\0';
128         return (name_read + nread);
129     }

```

Εκτός των άλλων, υλοποιήθηκε αλλαγή ονόματος χρήστη που γίνεται με την εντολή “\name” από τον χρήστη και ακολουθείται το όνομα με το οποίο επιθυμεί να γίνεται ορατός στους υπόλοιπους χρήστες. Με την πληκτρολόγηση της συγκεκριμένης εντολής ο εξυπηρετητής δε λαμβάνει το μήνυμα ως κανονικό μήνυμα συνομιλίας αλλά ερμηνεύεται ως εντολή και αντικαθιστά το αποθηκευμένο όνομα του πελάτη με το νέο όνομα που ακολουθεί την εντολή. Σε

περίπτωση που ο χρήστης επιθυμήσει να αλλάξει εκ νέου το όνομα του μπορεί να το πραγματοποιήσει με την ίδια εντολή χωρίς να γίνει συγχώνευση το παλιό με το νέο όνομα. Ακολουθεί το συγκεκριμένο κομμάτι κώδικα που είναι μέρος της **receive_thread()** μέσα στο αρχείο **serverVersion3.c**:

```
37     if (strncmp("\\name ", msg, 6) == 0){
38         int i;
39         memset(client_name, 0, MAX_CLIENT_NAME_LEN+1); //Clear client's name before change
40         for (i = 0; i < (MAX_CLIENT_NAME_LEN+1) && (msg[i+6] != '\0'); i++){
41             client_name[i] = msg[i+6];
42         }
43         client_name[i] = '\0';
44         continue;
45     }
```

Για κάθε νέο πελάτη ο εξυπηρετητής δημιουργεί ένα νέο thread με την εντολή **pthread_create()** ώστε το thread να λαμβάνει μηνύματα από τον συγκεκριμένο πελάτη, να ανιχνεύει αν ο πελάτης αποσυνδέθηκε και να μεταδίδει τα μηνύματα στους υπόλοιπους συνδεδεμένους πελάτες με τη συνάρτηση **broadcastClientsList()**. Η προαναφερθείσα συνάρτηση διατρέπει όλους τους συνδεδεμένους πελάτες και στέλνει το μήνυμα σε όλους εκτός από τον αποστολέα. Η υλοποίηση της **broadcastClientList()** στο αρχείο **clientsList.c**:

```
49 int broadcastClientsList(struct clientsList *cl, const char *msg, const char *sender, int sender_fd) {
50
51     pthread_mutex_lock(&cl->clients_lock);
52     for(int i = 0; i < cl->clients_num; i++){
53         int fd = cl->clients_fd[i];
54         if(fd != sender_fd){
55             if(sendNewMessage(fd, msg, sender) <= 0){
56                 perror("Failed to sendNewMessage");
57             }
58         }
59     }
60     pthread_mutex_unlock(&cl->clients_lock);
61
62     return 0;
63 }
```

Κώδικας για Πελάτη:


```

15 struct infoForReceiver{
16     int* socket_fd;
17     int* EOF_found;
18     char* msg;
19 };
20
21 void* receive_thread(void* infoForReceiver){
22     int* socket_fd = ((struct infoForReceiver*)infoForReceiver)->socket_fd;
23     int* EOF_found = ((struct infoForReceiver*)infoForReceiver)->EOF_found;
24     while (!(*EOF_found)){
25         char msg[MAX_MSG_LEN + 1];
26         char print_msg[MAX_PRINT_MSG_LEN + 1];
27         char name[MAX_CLIENT_NAME_LEN + 1];
28         memset(name, 0, MAX_CLIENT_NAME_LEN+1);
29         memset(msg, 0, MAX_MSG_LEN+1);
30         memset(print_msg, 0, MAX_PRINT_MSG_LEN + 1);
31         ssize_t read_bytes = recvNewMessage(*socket_fd, msg, MAX_MSG_LEN, name, MAX_CLIENT_NAME_LEN);
32         if (read_bytes <= 0){
33             if (*EOF_found == 1)
34                 fprintf(stdout, "I have disconnected\n");
35             else{
36                 fprintf(stdout, "Server disconnected! Terminating\n");
37                 close(*socket_fd);
38                 exit(EXIT_FAILURE);
39             }
40             break;
41         }
42         if (!(*EOF_found))
43             snprintf(print_msg, sizeof(print_msg), "\033[35m%s\033[0m> %s\n> ", name, msg);
44         printMsg(stdout, print_msg);
45         strcpy(msg, "");
46     }
47     return NULL;
48 }

```

```

48 int main()
49 {
50     struct sockaddr_in addr;
51     int socket_fd;
52     char msg[MAX_MSG_LEN + 1];
53     char print_msg[MAX_PRINT_MSG_LEN + 1];
54     int EOF_found;
55     struct infoForReceiver infoForReceiver;
56
57     socket_fd = socket(AF_INET, SOCK_STREAM, 0);
58     if (socket_fd == -1) {
59         perror("socket failure");
60         exit(EXIT_FAILURE);

```



```

78     EOF_found = 0;
79     pthread_t receiver; //New thread for receiving messages
80
81     infoForReceiver.socket_fd = &socket_fd;
82     infoForReceiver.EOF_found = &EOF_found;
83     infoForReceiver.msg = msg;
84     if (pthread_create(&receiver, NULL, receive_thread, &infoForReceiver ) != 0){
85         fprintf(stderr, "Failed at creating thread for receiving messages. Terminating client.");
86         exit(EXIT_FAILURE);
87     }
88
89     do {
90         do {
91             snprintf(print_msg, sizeof(print_msg), "> ");
92             printMsg(stdout, print_msg);
93
94             if (fgets(msg, sizeof(msg), stdin) == NULL) {
95                 EOF_found = 1;
96                 shutdown(socket_fd, SHUT_RDWR); //make receiver thread move on
97                 break;
98             }
99             msg[strcspn(msg, "\n")] = '\0';
100         } while (strlen(msg) == 0 && !EOF_found);
101
102         if (!EOF_found) {
103             if (sendMessage(socket_fd, msg) == -1) {
104                 for (int i = 0; i < 3;){
105                     if (sendMessage(socket_fd, msg) == -1)
106                         i++;
107                     else break;
108                     if (i == 2){
109                         perror("sendMessage failure");
110                         exit(EXIT_FAILURE);
111                     }
112                 }
113             }
114         }
115     } while (!EOF_found);
116     if (pthread_join(receiver, NULL) != 0){
117         fprintf(stderr, "Failed at joining receiver thread with main thread. Terminating client.");
118         close(socket_fd);
119         exit(EXIT_FAILURE);
120     }
121     close(socket_fd);
122     return 0;
123 }

```

Υλοποίηση εξυπηρετητή

Κατά την υλοποίηση χρησιμοποιήθηκε δυναμική δέσμευση μνήμης (malloc, calloc) για τα file descriptors και τις δομές που μεταφέρονται στα νήματα. Για κάθε δέσμευση μνήμης υπάρχει αντίστοιχη αποδέσμευση με free() ώστε να αποφεύγονται memory leaks. Επίσης, μετά από κάθε malloc προστέθηκε έλεγχος για το αν επέστρεψε null ώστε σε περίπτωση αποτυχίας δέσμευσης να τερματίζεται σωστά η σύνδεση χωρίς να συνεχίζεται η εκτέλεση.

Ο εξυπηρετητής τερματίζεται αυτόματα όταν αποσυνδεθεί και ο τελευταίος πελάτης. Για να υλοποιηθεί αυτό κάθε φορά που ένας πελάτης αποσυνδέεται αφαιρείται από την λίστα πελατών μέσω της `removeClient()`. Έπειτα, ελέγχεται ο αριθμός των συνδεδεμένων πελατών μέσω της μεταβλητής `clients_num`. Αν η τιμή της γίνει 1 σημαίνει ότι έχουμε μόνο έναν ενεργό πελάτη και ο εξυπηρετητής τερματίζει όταν αποσυνδέεται και αυτός. Πριν τον τερματισμό όμως, κλείνει η υποδοχή του τελευταίου πελάτη, αναμένει το κυρίως νήμα να τερματίσει τη λειτουργία του το τελευταίο νήμα που εξυπηρετούσε πελάτη, αποδεσμεύονται οι δομές της λίστας πελατών, και κλείνει η `listening socket` του εξυπηρετητή. Με αυτό τον τρόπο εξασφαλίζεται ότι δεν παραμένουν ανοιχτές υποδοχές και ενεργά νήματα, καθώς και πως ο εξυπηρετητής δε συνεχίζει να περιμένει νέες συνδέσεις όταν δεν υπάρχουν πελάτες.

Ακολουθεί ο κώδικας του εξυπηρετητή:

```
15 void* receive_thread(void* clientInfo){
16     struct clientInfoPassed * infoPassed = (struct clientInfoPassed*) clientInfo;
17     int *client_fd = infoPassed->client_fd;
18     int *socket_fd = infoPassed->socket_fd;
19     struct clientsList* allClients = infoPassed->allClients;
20     struct clientInfoPassed*** structureWithAllInfoPassed = infoPassed->structureWithAllInfoPassed;
21     char msg[MAX_MSG_LEN + 1];
22     char* client_name;
23     client_name = (char *) calloc((MAX_CLIENT_NAME_LEN+1), sizeof(char));
24     if (client_name == NULL){
25         fprintf(stderr, "Allocating memory for client with file descriptor %ls failed. Terminating connection\n", client_fd);
26         removeReceiverThreadInfo(structureWithAllInfoPassed, &infoPassed);
27         return NULL;
28     }
29     char client_filedescriptorForDeafaultName[64];
30     sprintf(client_filedescriptorForDeafaultName, "%d", ((*client_fd) - 3)); //Make a client have a default name
31     strcat(client_name, client_filedescriptorForDeafaultName);
32     fprintf(stdout, "%s has connected\n", client_name);
33     while (1){
34         memset(msg, 0, MAX_MSG_LEN+1);
35         ssize_t read_bytesNew = recvMessage(*client_fd, msg, MAX_MSG_LEN); //The last variable might not be MAX_MSG_LEN and m
36         char print_msg[MAX_PRINT_MSG_LEN + 1];
37         if (strncmp("\\name ", msg, 6) == 0){
38             int i;
39             memset(client_name, 0, MAX_CLIENT_NAME_LEN+1); //Clear client's name before change
40             for (i = 0; i < (MAX_CLIENT_NAME_LEN+1) && (msg[i+6] != '\\0'); i++){
41                 client_name[i] = msg[i+6];
42             }
43             client_name[i] = '\\0';
44             continue;
45         }
46         if (read_bytesNew > 0) {
47             broadcastClientsList(allClients, msg, client_name, *client_fd); //We want to broadcast the message that has been r
48         }
49         if (read_bytesNew == 0) {
50             snprintf(print_msg, sizeof(print_msg), "-- User %s has left the conversation\n", client_name);
51             printMsg(stdout, print_msg);
52             char stringForOthers[35+MAX_CLIENT_NAME_LEN];
```

```

52 char stringForOthers[35+MAX_CLIENT_NAME_LEN];
53 strncpy(stringForOthers, "--User ", 8);
54 strcat(stringForOthers, client_name);
55 strcat(stringForOthers, " has left the conversation");
56 broadcastClientsList(allClients, stringForOthers, client_name, *client_fd);
57 pthread_mutex_lock(&(allClients->clients_lock));
58 int numOfClients = allClients->clients_num;
59 fprintf(stdout, "Number of clients is %d\n", (numOfClients-1));
60 if (numOfClients == 1){
61     pthread_mutex_unlock(&(allClients->clients_lock));
62     shutdown(*socket_fd, SHUT_RDWR); //Make main thread move on
63     printf("No more clients to serve. Terminating... \n");
64 }else{
65     pthread_mutex_unlock(&(allClients->clients_lock));
66     removeReceiverThreadInfo(structureWithAllInfoPassed, &infoPassed);
67 }
68 free(client_name);
69 return NULL;
70 }
71 else if (read_bytesNew == -1) {
72     perror("recvMessage failure");
73     close(*client_fd);
74     free(client_name);
75     removeReceiverThreadInfo(structureWithAllInfoPassed, &infoPassed);
76     return NULL;
77 } else if (read_bytesNew == -2) {
78     snprintf(print_msg, sizeof(print_msg), "error: the message of the client cannot fit in the 'msg' buffer\n");
79     printMsg(stderr, print_msg);
80     close(*client_fd);
81     free(client_name);
82     removeReceiverThreadInfo(structureWithAllInfoPassed, &infoPassed);
83     return NULL;
84 }
85 snprintf(print_msg, sizeof(print_msg), "\033[35mUser %s\033[0m> %s\n> ", client_name, msg);
86 printMsg(stdout, print_msg);
87 strcpy(msg, "");
88 }
89 }

```

```

91 int main() {
92     struct sockaddr_in addr;
93     int socket_fd;
94     int socket_option;
95
96     struct sockaddr_in client_addr;
97     socklen_t client_addr_len = sizeof(client_addr);
98     struct clientInfoPassed** AllReceiverThreadsInfo;
99
100     AllReceiverThreadsInfo = (struct clientInfoPassed **) malloc(MAX_CONNECTED_CLIENTS * sizeof(struct clientInfoPassed *)); //Allocate memory
101     //in which every information necessary for receiver threads is stored
102     if (AllReceiverThreadsInfo == NULL){
103         fprintf(stderr, "Allocating memory for structure to store all information for receiver threads failed. Terminating\n");
104         return 0;
105     }
106
107     initializeAllClientsInfo(&AllReceiverThreadsInfo);

```

```

139 struct clientsList clients; //List of clients
140 initClientsList(&clients); //Intitializing list of clients
141 while (1) {
142     int *client_fd;
143     client_fd = (int *) malloc(sizeof(int));
144     if (client_fd == NULL){
145         fprintf(stderr, "Allocating memory to store a client's file descriptor failed. Retrying\n");
146         continue;
147     }
148
149     *client_fd = accept(socket_fd, (struct sockaddr *)&client_addr, &client_addr_len);
150     if (*client_fd == -1){
151         if (numberOfRemainingClients(&AllReceiverThreadsInfo) == 1) break;
152         fprintf(stderr, "Accepting client failed\n");
153         continue;
154     }
155     if (addClient(&clients, *client_fd) == -1){
156         perror("accept failure. no available space for new clients. try later");
157         free(client_fd);
158         break;
159     }
160
161     pthread_mutex_lock(&clients.clients_lock);
162     fprintf(stdout, "New client connected number of clients is %d\n", clients.clients_num);
163     pthread_mutex_unlock(&clients.clients_lock);
164
165     struct clientInfoPassed * infoAboutClients;
166     infoAboutClients = (struct clientInfoPassed *) malloc(sizeof(struct clientInfoPassed));
167     if (infoAboutClients == NULL){
168         fprintf(stderr, "Allocating memory to store info for client failed\n");
169         close(*client_fd);
170         free(client_fd);
171         continue;
172     }
173     infoAboutClients->client_fd = (int *) malloc(sizeof(int));
174     if (infoAboutClients->client_fd == NULL){
175         fprintf(stderr, "Allocating memory to store info for client's file descriptor failed\n");
176         close(*client_fd);

```



```

176     close(*client_fd);
177     free(client_fd);
178     free(infoAboutClients);
179     continue;
180 }
181 memcpy(infoAboutClients->client_fd, client_fd, sizeof(int));
182 infoAboutClients->allClients = &clients;
183 infoAboutClients->socket_fd = &socket_fd;
184 infoAboutClients->threadAddress = (pthread_t *) malloc(sizeof(pthread_t));
185 if (infoAboutClients->threadAddress == NULL){
186     fprintf(stderr, "Allocating memory to store info for client failed\n");
187     close(*client_fd);
188     free(client_fd);
189     free(infoAboutClients->client_fd);
190     free(infoAboutClients);
191     continue;
192 }
193 infoAboutClients->structureWithAllInfoPassed = &AllReceiverThreadsInfo;
194 addReceiverThreadInfo(&AllReceiverThreadsInfo, &infoAboutClients);
195 if (pthread_create((infoAboutClients->threadAddress), NULL, receive_thread, infoAboutClients) != 0){
196     fprintf(stderr, "Creating thread %p to handle client with file descriptor %ls failed\n", infoAboutClients->threadAddress, client_fd);
197     removeReceiverThreadInfo(&AllReceiverThreadsInfo, &infoAboutClients);
198     continue;
199 }
200 if (pthread_detach(*(infoAboutClients->threadAddress)) != 0){
201     fprintf(stderr, "Detaching thread %p which handles client with file descriptor %ls failed\n", infoAboutClients->threadAddress, client_fd);
202     close(*(infoAboutClients->client_fd));
203     removeReceiverThreadInfo(&AllReceiverThreadsInfo, &infoAboutClients);
204     continue;
205 }
206 free(client_fd);
207 }

```

```

209 pthread_t * lastReceiverThread = NULL;
210 struct clientInfoPassed* lastRemainingClientInfo;
211
212 for (int i = 0; i<MAX_CONNECTED_CLIENTS ;i++){
213     if (AllReceiverThreadsInfo[i] == NULL){
214         continue;
215     }
216     lastReceiverThread = AllReceiverThreadsInfo[i]->threadAddress;
217     lastRemainingClientInfo = AllReceiverThreadsInfo[i];
218 }
219
220 pthread_join(*lastReceiverThread, NULL);
221 removeReceiverThreadInfo(&AllReceiverThreadsInfo, &lastRemainingClientInfo);
222 destroyAllClientsInfo(&AllReceiverThreadsInfo); //Destroying list of clients
223 close(socket_fd); //Closing socket file descriptor
224 return 0;
225 }

```

ΚΕΦΑΛΑΙΟ 3: Τελικά αποτελέσματα στο τερματικό

Παρακάτω παραθέτουμε στιγμιότυπα οθόνης από την κάθε έκδοση της εργασίας όπου φαίνεται η χρήση και η λειτουργικότητα του συστήματος.

- Έκδοση#1: αίτηση-απάντηση με ένα νήμα ανά διεργασία Απλό παράδειγμα

με αποσύνδεση του server:

```
cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./serverVersion1
Listening on port 8080
-- User 127.0.0.1 has joined the conversation
User 127.0.0.1> Thelo na syndetho sto logariasmo mou
Reply> Syndethikate me epitychia sto logariasmo sas
User 127.0.0.1> Thelo na kano katathesi 1000000$
Reply> H katathesi pragmatopoiithike me epitychia
User 127.0.0.1> Thelo na aposyndetho apo to logariasmo mou
Reply> H aposynthesi egine me epitychia
^C
cs205101@opti313:~/Katanemhmena/FINAL_CODE$
```

```
cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./clientVersion1
> Thelo na syndetho sto logariasmo mou
Server> Syndethikate me epitychia sto logariasmo sas
> Thelo na kano katathesi 1000000$
Server> H katathesi pragmatopoiithike me epitychia
> Thelo na aposyndetho apo to logariasmo mou
Server> H aposynthesi egine me epitychia
> ^C
cs205101@opti313:~/Katanemhmena/FINAL_CODE$
```

Παράδειγμα που αποσυνδέεται ο χρήστης:

```
> cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./clientVersion1
> kalimera
Server>
> cs205101@opti313:~/Katanemhmena/FINAL_CODE$

cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./serverVersion1
Listening on port 8080
-- User 127.0.0.1 has joined the conversation
User 127.0.0.1> kalimera
-- User 127.0.0.1 has left the conversation
cs205101@opti313:~/Katanemhmena/FINAL_CODE$
```

- Έκδοση#2: αίτηση-απάντηση με δύο νήματα ανά διεργασία

Παράδειγμα με βίαιη αποσύνδεση του πελάτη:

```
cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./clientVersion2
> Thelo na syndetho sto loagariasmo mou
> Thelo na kano katathesi 100000$
Server> Exete katachorysei lanthas,ena stoixeia
Server> De mporeite na kanete katathesi
> Thelo na syndetho ek neoy sto logariasmo mou
Server> Syndethikate me epitychia sto logariasmo sas
> Thelo na kano katathesi 100000$
> Thelo na pliroso th DEI
> ^C
cs205101@opti313:~/Katanemhmena/FINAL_CODE$
```

```
cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./serverVersion2
Listening on port 8080
-- User 127.0.0.1 has joined the conversation
User 127.0.0.1> Thelo na syndetho sto loagariasmo mou
User 127.0.0.1> Thelo na kano katathesi 100000$
> Exete katachorysei lanthas,ena stoixeia
Reply> De mporeite na kanete katathesi
User 127.0.0.1> Thelo na syndetho ek neoy sto logariasmo mou
> Syndethikate me epitychia sto logariasmo sas
User 127.0.0.1> Thelo na kano katathesi 100000$
User 127.0.0.1> Thelo na pliroso th DEI
-- User 127.0.0.1 has left the conversation
Client disconnected. Terminating
cs205101@opti313:~/Katanemhmena/FINAL_CODE$
```

Παράδειγμα με απλή αποσύνδεση του πελάτη:

```
cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./serverVersion2
Listening on port 8080
-- User 127.0.0.1 has joined the conversation
User 127.0.0.1> Kalimera
User 127.0.0.1> Ti kaneis
> Ola kala
-- User 127.0.0.1 has left the conversation
Client disconnected. Terminating
cs205101@opti313:~/Katanemhmena/FINAL_CODE$
```

```
cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./clientVersion2
> Kalimera
> Ti kaneis
Server> Ola kala
> Terminating connection... Exiting
cs205101@opti313:~/Katanemhmena/FINAL_CODE$
```

Παράδειγμα με αποσύνδεση του server:

```

cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./serverVersion2
Listening on port 8080
-- User 127.0.0.1 has joined the conversation
User 127.0.0.1> Kalimera
>
cs205101@opti313:~/Katanemhmena/FINAL_CODE$ █

cs205101@opti313:~/Katanemhmena/FINAL_CODE$ ./clientVersion2
> Kalimera
> Server disconnected.
cs205101@opti313:~/Katanemhmena/FINAL_CODE$ █

```

- Έκδοση #3: αίτηση χωρίς απάντηση με πολυνηματικό εξυπηρετητή

Παράδειγμα με βίαιη απόσυνδεση του server:

Στα αριστερά είναι ο εξυπηρετητής και στα δεξιά ένας πελάτης:

<pre> cs205101@opti313:~/Katanemhmena/final_final_code\$./serverVersion3 Listening on port 50505. New client connected number of clients is 1 1 has connected New client connected number of clients is 2 2 has connected User Greg> Geia User 2> Geia User Dimitris> Geia User Greg> Pos elsai User Dimitris> Kala User Dimitris> Esy User Dimitris> ? User Greg> kala ki ego User Greg> pos paei h ergasia User Dimitris> Proxora!!! > ^C cs205101@opti313:~/Katanemhmena/final_final_code\$ █ </pre>	<pre> cs205101@opti313:~/Katanemhmena/final_final_code\$./clientVersion3 > \name Greg > Geia 2> Geia Dimitris> Geia > Pos elsai Dimitris> Kala Dimitris> Esy Dimitris> ? > > kala ki ego > pos paei h ergasia Dimitris> Proxora!!! > Server disconnected! Terminating cs205101@opti313:~/Katanemhmena/final_final_code\$ █ </pre>
---	---

Άλλος ένας πελάτης του παραδείγματος που συνδέεται στον εξυπηρετητή ταυτόχρονα με τον πελάτη που φαίνεται στην από πάνω εικόνα:

```

cs205042@opti321:~/Desktop/final_final_code$ ./clientVersion3
Greg> Geia
>
> \name Dimitris
> Geia
Greg> Pos elsai
> Kala
> Esy
> ?
Greg> kala ki ego
Greg> pos paei h ergasia
> Proxora!!!
> Server disconnected! Terminating
cs205042@opti321:~/Desktop/final_final_code$ █

```

Παράδειγμα με αποσύνδεση πελάτη:

Στα αριστερά είναι ο εξυπηρετητής και στα δεξιά ένας πελάτης:

Άλλος ένας πελάτης συνδεδεμένος την ίδια στιγμή:

```
User Dimitris> Proxora!!!
> ^C
cs205101@opti313:~/Katanemhmena/final_final_code$ ./serverVersion3
Listening on port 50505.
New client connected number of clients is 1
1 has connected
New client connected number of clients is 2
2 has connected
User 2> hola
User Greg> Buanasera
User Dimitris> Ti?
User Greg> kalispera sta italika
-- User Dimitris has left the conversation
Number of clients is 1
[]

cs205042@opti321:~/Desktop/final_final_code$ ./clientVersion3
> hola
> \name Dimitris
Greg> Buanasera
> Ti?
Greg> kalispera sta italika
> I have disconnected
cs205042@opti321:~/Desktop/final_final_code$ []
```

Παράδειγμα με αποσύνδεση όλων των πελατών:

Η δομή των εικόνων είναι όμοια με την από πάνω:

Παράδειγμα στο οποίο ένας πελάτης αποσυνδέεται βίαια:

```
New client connected number of clients is 2
2 has connected
User 2> hola
User Greg> Buanasera
User Dimitris> Ti?
User Greg> kalispera sta italika
-- User Dimitris has left the conversation
Number of clients is 1
New client connected number of clients is 2
3 has connected
User Kala> Ti na kano?
User Greg> kalos hrthes xana
User Greg> pali vrexhei
User Kala> Nai, vrexel
-- User Greg has left the conversation
Number of clients is 1
-- User Kala has left the conversation
Number of clients is 0
No more clients to serve. Terminating...
cs205101@opti313:~/Katanemhmena/final_final_code$ []

Dimitris> --User Dimitris has left the conversation
Kala> Ti na kano?
> kalos hrthes xana
> pali vrexhei
Kala> Nai, vrexel
> I have disconnected
cs205101@opti313:~/Katanemhmena/final_final_code$ []

cs205042@opti321:~/Desktop/final_final_code$ ./clientVersion3
> \name Kala
> Ti na kano?
Greg> kalos hrthes xana
Greg> pali vrexhei
> Nai, vrexel
Greg> --User Greg has left the conversation
> I have disconnected
cs205042@opti321:~/Desktop/final_final_code$ []
```

Στα αριστερά είναι ο εξυπηρετητής και στα δεξιά ένας πελάτης:

```

cs205101@opti313:~/Katanemhmena/final_final_code$ ./serverVersion3
Listening on port 50505.
New client connected number of clients is 1
1 has connected
New client connected number of clients is 2
2 has connected
User 2> Exei vgalei hlio
User Kala> Ta pramata proxorane
-- User Kala has left the conversation
Number of clients is 1

```

```

cs205101@opti313:~/Katanemhmena/final_final_code$ ./clientVersion3
2> Exei vgalei hlio
Kala> Ta pramata proxorane
Kala> --User Kala has left the conversation
>

```

Άλλος ένας πελάτης:

```

cs205042@opti321:~/Desktop/final_final_code$ ./clientVersion3
> Exei vgalei hlio
> \name Kala
> Ta pramata proxorane
> ^C
cs205042@opti321:~/Desktop/final_final_code$

```